

Pandas Data Selection: loc[] vs iloc[]

These are the **two most important indexing methods** in Pandas.

Method	Full Form	Uses
loc[]	Location	Select data using labels (names)
iloc[]	Integer Location	Select data using positions (numbers)

First Understand the DataFrame

Suppose we have:

```
import pandas as pd
```

```
df = pd.DataFrame({  
    "foo": ["A", "B", "C", "A", "B", "C"],  
    "bar": [1, 2, 3, 4, 5, 6],  
    "baz": ["x", "y", "z", "q", "w", "t"]  
},  
index=["zero", "one", "two", "three", "four", "five"])
```

```
print(df)
```

Output:

Index Label	foo	bar	baz
zero	A	1	x
one	B	2	y
two	C	3	z
three	A	4	q
four	B	5	w
five	C	6	t

1. loc[] (Label Based Selection)

loc[] works with:

- Row labels
- Column labels

Syntax

```
df.loc[row_label, column_label]
```

Think:

loc = Label

Select Single Row

```
df.loc["zero"]
```

Output:

```
foo  A
bar  1
baz  x
```

Because "zero" is the row label.

Select Multiple Rows

```
df.loc[["zero", "two"]]
```

Output:

	foo	bar	baz
zero	A	1	x
two	C	3	z

Select Specific Row and Column

```
df.loc["zero", "foo"]
```

Output:

'A'

Select Multiple Columns

```
df.loc["zero", ["foo", "baz"]]
```

Output:

```
foo    A  
baz    x
```

Slicing with loc

```
df.loc["zero":"two"]
```

Output:

	foo	bar	baz
zero	A	1	x
one	B	2	y
two	C	3	z

Important

In loc[]

✅ Ending value INCLUDED

```
df.loc["zero":"two"]
```

includes:

```
zero  
one  
two
```

2. `iloc[]` (Integer Position Based Selection)

Works with row numbers.

Syntax

```
df.iloc[row_position, column_position]
```

Think:

`iloc` = Integer Location

Select First Row

```
df.iloc[0]
```

Output:

```
foo  A
bar   1
baz   x
```

Position 0 = first row.

Select Third Row

```
df.iloc[2]
```

Output:

```
foo  C
bar   3
baz   z
```

Select Multiple Rows

```
df.iloc[[0,2]]
```

Output:

	foo	bar	baz
zero	A	1	x
two	C	3	z

Select Specific Cell

```
df.iloc[0,0]
```

Output:

'A'

Explanation:

Row 0 = zero

Column 0 = foo

Value = A

Select Multiple Columns

```
df.iloc[0,[0,2]]
```

Output:

foo A

baz x

Slicing with iloc

```
df.iloc[0:2]
```

Output:

	foo	bar	baz
zero	A	1	x
one	B	2	y

Important

In `iloc[]`

✗ Ending position EXCLUDED

```
df.iloc[0:2]
```

includes:

0

1

NOT

2

Exactly like Python lists.

Most Important Difference

loc

```
df.loc["zero":"two"]
```

Result:

zero

one

two

End Included 

iloc

```
df.iloc[0:2]
```

Result:

0

1

End Excluded 

Selecting Rows and Columns Together

Using loc

```
df.loc["one":"three", ["foo", "baz"]]
```

Output:

	foo	baz
one	B	y
two	C	z
three	A	q

Using iloc

```
df.iloc[1:4, [0,2]]
```

Output:

	foo	baz
one	B	y
two	C	z
three	A	q

Conditional Selection with loc

Very commonly used in Data Science.

Get rows where bar > 3

```
df.loc[df["bar"] > 3]
```

Output:

	foo	bar	baz
three	A	4	q
four	B	5	w
five	C	6	t

Real Dataset Example

Suppose:

```
df.head()
```

	Name	Age	City
0	Arpan	20	Patan
1	Raj	21	Ahmedabad
2	Jay	19	Surat

loc

```
df.loc[1]
```

If index label is 1.

iloc

`df.iloc[1]`

Second row by position.

Why Both Exist?

Suppose:

`df.index = [10,20,30]`

Data:

Index	Name
10	A
20	B
30	C

loc

`df.loc[20]`

Returns row with label 20.

iloc

`df.iloc[1]`

Returns second row.

Even though label is 20.

This is the biggest reason both methods exist.